

COMPUTER ORGANIZATION AND ARCHITECTURE

UNIT IV – MEMORY ORGANIZATION & I/O ORGANIZATION

Short Notes | Exam Ready

1. MEMORY HIERARCHY

Memory hierarchy is a **multi-level arrangement of storage** in a computer, ranging from the fastest (but smallest and costliest) at the top to the slowest (but largest and cheapest) at the bottom.

Levels (Top → Bottom):

Level	Type	Speed	Cost	Capacity
1	CPU Registers	Fastest	Highest	Smallest (64/128 bits)
2	Cache Memory	Very Fast	High	Small (KB–MB)
3	Main Memory (RAM)	Fast	Moderate	Medium (GB)
4	Magnetic Disk	Slow	Low	Large (TB)
5	Magnetic Tape	Slowest	Lowest	Largest

Key Characteristics:

- **Capacity:** Increases as you go down the hierarchy (tape > disk > RAM > cache > register).
- **Access Time:** Time taken to read/write data. Increases going down (registers are fastest).
- **Cost per bit:** Decreases going down (registers are most expensive per bit).
- **Performance:** Frequently used data is stored in faster levels to reduce retrieval time.

Multiprogramming:

- Running **multiple programs at the same time** on one CPU.
- Programs too large for main memory are kept in auxiliary (secondary) memory.
- The **Memory Management System** supervises transfer of programs between auxiliary and main memory.

2. MAIN MEMORY

Main memory is the **primary storage** of a computer. It is directly accessible by the CPU and stores the programs and data currently in use. It uses **semiconductor integrated circuits**.

RAM – Random Access Memory:

- **SRAM (Static RAM):** Stores data using flip-flops (tiny circuits that hold 0 or 1). Retains data as long as power is on. Extremely fast but very expensive. Used in **Cache Memory**.
- **DRAM (Dynamic RAM):** Stores data as electric charge in tiny capacitors. Needs regular refreshing. Slower than SRAM but cheaper and has **higher capacity**. Used in **Main Memory**.

ROM – Read Only Memory:

- Stores **permanent programs** that don't change, like the bootstrap loader.
- **Bootstrap Loader:** The first program that runs when a computer is switched on. It loads the operating system (OS) from the disk into main memory.
- ROM is **non-volatile** – it retains data even without power.

RAM Chip:

- Has a **bidirectional data bus** – data flows in both directions (read & write).
- Uses **three-state buffers** – output can be logic 0, logic 1, or High Impedance (disconnected).
- Has **Chip Select (CS)** inputs to enable/disable the chip, and **RD/WR** inputs for read/write operations.

ROM Chip:

- Data bus is **output only** (no write operation).
- No RD/WR control needed. Only CS inputs are required.

Memory Address Map:

A **Memory Address Map** is a table that shows which range of addresses is assigned to each chip (RAM or ROM) in the system. It helps the CPU know which chip to select for a given address.

- Address bus line 10 = 0 → selects RAM; line 10 = 1 → selects ROM.
- A 2×4 decoder is used to select one of the four RAM chips based on address lines 8 and 9.

3. AUXILIARY MEMORY (Secondary Memory)

Auxiliary memory is also called **secondary memory or non-volatile memory**. It stores programs and data for the long term and is **NOT directly accessed by the CPU**. It has lower cost and higher capacity than main memory.

Key Terms:

- **Access Time:** Average time to reach a storage location and get its contents. Includes seek time + transfer time.
- **Seek Time:** Time to position the read/write head over the correct track.
- **Transfer Rate:** Number of characters/words the device can transfer per second once positioned.
- **Record:** A fixed number of characters or words. Reading/writing is always done on entire records.

Magnetic Disk:

- A **circular plate** coated with magnetic material. Both sides can be used. Multiple disks can be stacked on a spindle.
- Data is stored in **tracks** (concentric circles on the disk surface). Tracks are divided into **sectors**.
- A disk address specifies: disk number, disk surface, sector number, track within sector.
- **Hard Disk:** Permanently attached to the drive assembly, cannot be removed by the user.
- **Floppy Disk:** Removable disk made of plastic coated with magnetic material. Common sizes: 5.25 inch and 3.5 inch.

Magnetic Tape:

- A strip of plastic coated with magnetic recording material.
- Bits are stored as **magnetic spots** along several tracks simultaneously.
- Usually **7 or 9 bits** are recorded at once to form one character (including a parity bit for error checking).
- Can be stopped, started, moved forward or in reverse, and rewound.
- Each record on tape has an identification bit pattern at the beginning and end.

4. ASSOCIATIVE MEMORY / CONTENT ADDRESSABLE MEMORY (CAM)

In normal RAM, data is found by its **address (location)**. In Associative Memory (CAM), data is found by its **content (what it contains)**. All words in the memory are searched **simultaneously and in parallel**.

Key Points:

- When **writing** to associative memory, no address is given – the memory finds an empty location itself.
- When **reading**, you provide the content (or part of it) and matching words are returned.
- More **expensive than RAM** because each cell has both storage and comparison logic circuits.
- Used where **search speed is critical** (e.g., network routers, database lookups).

Hardware Organization:

- **Argument Register (A):** Holds the data/content to be searched.
- **Key Register (K):** A mask that selects which bits of A to compare. If K-bit = 1, that bit is compared; if K-bit = 0, it is ignored.
- **Match Register (M):** Has one bit per word. A bit is set to 1 if the corresponding word matches the argument.
- **Memory Array:** Stores m words, each of n bits.

Match Logic Formula:

For word i to match argument A (ignoring masked bits via K):

→ Each bit position j is checked: $x_j = 1$ if A_j equals the stored bit F_{ij} (both same), else 0.

→ If $K_j = 0$ (masked/ignored), that bit position is treated as a match automatically.

→ Match bit $M_i = 1$ only if ALL bit positions produce a match (AND of all terms).

Applications:

- Database search acceleration

- Network routing (finding correct path for data packets)
- Image processing (pattern search)

Advantages vs Disadvantages:

Advantages	Disadvantages
Very fast parallel search	More expensive than RAM
Suitable for high-speed lookups	Each cell needs extra logic circuits
Searches multiple items simultaneously	Complex to build

5. CACHE MEMORY

Cache is a **small, very high-speed memory** placed between the CPU and Main Memory. It stores **frequently used data** so the CPU can access it quickly without waiting for the slower main memory.

Basic Operation:

- **Cache Hit:** CPU looks for data → data IS found in cache → data is read directly from cache (fast).
- **Cache Miss:** CPU looks for data → data is NOT in cache → CPU fetches it from main memory → copies a block of data into cache for future use.
- When cache is full and new data must be loaded, an old block is removed using a **Replacement Algorithm**.

Hit Ratio (Performance Measure):

- **Hit Ratio = Number of Hits / Total Memory Accesses**
- **Miss Ratio = Number of Misses / Total Memory Accesses**
- Higher hit ratio = better cache performance = CPU spends less time waiting.

Replacement Algorithms (when cache is full):

Algorithm	Description
LRU (Least Recently Used)	Removes the block that has not been accessed for the longest time. Most commonly used.
FIFO (First-In First-Out)	Removes the oldest block in the cache, regardless of how often it is used.
LFU (Least Frequently Used)	Removes the block that has been accessed the fewest number of times.

Advantages:

- Faster than main memory
- Reduces CPU access time
- Stores most recently used data

Disadvantages:

- Limited capacity
- Very expensive compared to RAM
- Complex management
- Increased power consumption
- Potential data inconsistency

6. CACHE MEMORY MAPPING

Cache Mapping is a technique that defines how blocks of data from main memory are placed into cache memory. There are 3 types:

1. Associative Mapping:

- Any block from main memory can be placed in **any location** in cache.
- The cache stores both the **address (tag)** and **data** of each word.
- When CPU needs data, it places the address in the argument register and the entire cache is searched in parallel.
- **Advantage:** Most flexible, low conflict rate, more efficient.
- **Disadvantage:** Very expensive – needs complex comparison hardware.

2. Direct Mapping:

- Each block of main memory maps to **exactly one specific location** in cache.
- The 15-bit CPU address is split into two parts: **6-bit Tag + 9-bit Index**.
 - Index = used to find the cache location directly.
 - Tag = compared with the stored tag in that cache location to confirm it's the right data.
- **Hit:** Tags match → data found in cache.
- **Miss:** Tags don't match → fetch from main memory and replace cache entry.
- **Advantage:** Simple to implement, low cost.
- **Disadvantage:** High conflict rate – two addresses with same index but different tags can't be in cache at the same time, causing frequent misses.

3. Set-Associative Mapping:

- A **combination of direct and associative mapping**. Each cache location (set) can hold **2 or more blocks** from main memory.
- The CPU address index points to a set. Within that set, all stored tags are compared in parallel (associative search).
- Set size k = number of blocks stored per set. A cache with set size 2 stores 2 tag-data pairs per index.
- **Advantage:** Better hit ratio than direct mapping, less expensive than full associative mapping.
- **Disadvantage:** More complex logic than direct mapping, higher cost.

Comparison Summary:

Feature	Associative	Direct	Set-Associative
Flexibility	Highest	Lowest	Medium
Conflict Rate	Lowest	Highest	Medium
Cost/Complexity	Highest	Lowest	Medium
Hit Ratio	Best	Worst	Good

7. INPUT-OUTPUT (I/O) INTERFACE

I/O Interfaces are **special hardware components** placed between the CPU and peripheral devices (keyboard, disk, printer, etc.) to manage data transfer between them.

Why is I/O Interface Needed? (4 Reasons):

- Peripherals are **electromechanical** (slow, mechanical parts) while CPU is **electronic** (very fast) – signal conversion needed.
- Peripherals are **much slower** than CPU – a synchronization (timing) mechanism is needed.
- **Data formats differ** between peripherals and the CPU/memory.
- Different peripherals have different **operating modes** – each must be controlled independently.

I/O Commands:

Command	Purpose
Control Command	Activates the peripheral and tells it what to do (e.g., rewind tape).
Status Command	Checks the current status of the peripheral before data transfer begins.
Data Output Command	Transfers data from the CPU bus into the interface register (CPU → Device).
Data Input Command	Receives data from the peripheral and places it in the interface buffer (Device → CPU).

Functions of I/O Interface:

- **Speed Synchronization:** Matches the CPU's speed with the slower I/O device.
- **Processor Communication:** Accepts/decodes CPU commands and reports device status.
- **Signal Control:** Generates timing and control signals for smooth data transfer.
- **Data Buffering:** Temporarily holds data to manage speed differences between CPU and device.
- **Error Detection:** Detects transmission errors before they affect system performance.
- **Data Conversion:** Converts serial↔parallel data and digital↔analog signals.
- **Status Reporting:** Informs the CPU about the current state of the peripheral device.

Isolated I/O vs Memory-Mapped I/O:

Isolated I/O	No.	Memory-Mapped I/O
Uses separate address space for I/O	01	Uses same address space as memory
Special instructions: IN, OUT, INS, OUTS	02	Any memory instruction can be used
I/O device addresses called "ports"	03	I/O devices treated as memory locations
Efficient I/O operations (separate bus)	05	Less efficient (single bus for all)
Larger in size	06	Smaller in size
Complex internal logic	07	Common logic for memory and I/O
Slower operations	08	Faster operations

8. ASYNCHRONOUS DATA TRANSFER

When two units (like CPU and I/O device) operate with **independent clocks**, they are called **asynchronous**. Special techniques are needed to coordinate data transfer between them.

Method 1: Strobe Control

Uses a **single control signal (strobe pulse)** to tell the other unit when to read/write data.

- **Source-Initiated:** Source places data on bus, then activates strobe → destination reads data.
- **Destination-Initiated:** Destination activates strobe → source places data → destination reads it.
- **Disadvantage:** The initiating unit has **no way to confirm** the other unit actually received/sent the data. No acknowledgment.

Method 2: Handshaking

Uses **two control signals** – one from source to destination and one back from destination to source. This ensures both sides confirm the transfer.

- **Source-Initiated:** Source puts data on bus → enables 'Data Valid' signal → destination reads data → destination enables 'Data Accepted' → source disables 'Data Valid' → cycle ends.
- **Destination-Initiated:** Destination enables 'Ready for Data' → source places data → enables 'Data Valid' → destination reads and disables 'Ready for Data'.
- **Advantage:** More reliable – both units actively participate. Errors detected by **Timeout mechanism** (alarm if transfer not completed on time).

Asynchronous Serial Transmission:

Data is sent **one bit at a time** over a single wire. Each character is framed with special bits:

- **Start Bit (0):** Always 0. Signals the beginning of a character.
- **Character Bits:** The actual data bits (the character being transmitted).
- **Stop Bit (1):** Always 1. Signals the end of a character.
- Parallel transmission is faster but needs many wires. Serial is slower but cheaper and used for long distances.

Serial Transmission Implementations:

- **Asynchronous Communication Interface:** Acts as both transmitter and receiver. Has transmitter register, receiver register, control register, and status register. Converts parallel CPU data to serial for transmission and vice versa.
- **FIFO Buffer (First-In First-Out):** A memory unit where the first item stored is the first item retrieved. Has separate input/output terminals and can accept and deliver data at two different rates, useful when source is faster than destination.

9. MODES OF DATA TRANSFER

Data transfer between I/O devices and memory can happen in **3 ways**:

Mode 1: Programmed I/O

The CPU directly controls every data transfer. It **continuously checks (monitors)** the I/O device to see if it is ready.

- I/O device sets a **Flag bit (F)** in the status register to 1 when data is ready.

- CPU keeps checking the flag in a loop: Flag = 0 → keep checking; Flag = 1 → read data.
- CPU reads the data register and transfers data to memory. Then flag is cleared to 0.
3 steps per byte transfer: (1) Read status register, (2) Check flag, (3) Read data register.
- **Drawback:** CPU is stuck in a waiting loop (busy-wait) and cannot do other work. Very inefficient – CPU time is wasted just checking flags.

Mode 2: Interrupt-Initiated I/O

Instead of the CPU waiting, the I/O device **interrupts the CPU** when it is ready for transfer.

- CPU runs its normal program. When I/O device is ready, it sends an **Interrupt Request (INT)** signal to the CPU.
- CPU temporarily **stops its current program**, saves its state (on a stack), and runs the **Interrupt Service Routine (ISR)** to handle the I/O transfer.
- After ISR is done, CPU **resumes its original program** from where it left off.
- **Advantage over Programmed I/O:** CPU can do useful work between I/O operations.

Priority Interrupts:

- When **multiple devices interrupt simultaneously**, a **Priority Interrupt system** decides which is serviced first.
- **Software – Polling:** CPU checks each device one by one in order of priority. Simple but slow when many devices are present.
- **Hardware – Daisy Chaining:** Devices are connected in series. Device closest to CPU has highest priority. The interrupt acknowledge signal passes through the chain; a device with a pending interrupt blocks it from going further.
- **Hardware – Parallel Priority Interrupt:** Uses a priority encoder circuit. Each device has an interrupt register bit and a mask register bit. The encoder selects the highest priority active interrupt and sends its vector address (VAD) to the CPU.

Key Terms:

- **VAD (Vector Address):** Address of the ISR (Interrupt Service Routine) for each device.
- **NVAD (Non-Vector Address):** A fixed address where a common ISR is stored.
- **IEN (Interrupt Enable Flip-flop):** Master switch to enable/disable the entire interrupt system.
- **IST (Interrupt Status Flip-flop):** Set to 1 when a valid (unmasked) interrupt occurs.
- **INTACK:** Interrupt Acknowledge signal sent by CPU to confirm it received the interrupt.

Mode 3: Direct Memory Access (DMA)

DMA allows data to be transferred **directly between memory and I/O devices WITHOUT the CPU** acting as a middleman. This is the fastest I/O transfer method.

- A special hardware unit called the **DMA Controller** manages the transfer.
- During DMA transfer, the CPU is placed in an **idle (high-impedance) state** – it disconnects from the bus.
- **Bus Request (BR):** DMA sends this to CPU to request control of the bus.
- **Bus Grant (BG):** CPU sends this back to confirm DMA can take over the bus.
- After transfer is done, DMA releases the bus → CPU resumes.

DMA Controller Registers:

Register	Purpose
Address Register	Holds the memory address where data is to be read from or written to.
Word Count Register	Holds the number of words to be transferred. Decrement by 1 after each transfer. Transfer ends
Control Register	Specifies the mode of transfer (read/write direction, etc.).

DMA Transfer Modes:

- **Burst Transfer:** DMA takes full control of the bus and transfers an entire block of data continuously without releasing the bus. CPU is idle throughout.
- **Cycle Stealing:** DMA transfers only one word at a time, then releases the bus back to the CPU. CPU and DMA alternate access. Slower but CPU is not completely blocked.

I/O Transfer Modes at a Glance:

Feature	Programmed I/O	Interrupt I/O	DMA
CPU Involvement	Full – CPU always busy	Partial – CPU interrupted	None – CPU idle
Speed	Slowest	Moderate	Fastest
Efficiency	Very low	Good	Best
How it works	CPU polls flag	Device interrupts CPU	Controller directly accesses memory

Strobe vs Handshaking:

Strobe Control	Handshaking
Single control signal	Two control signals
No acknowledgment from receiver	Receiver sends acknowledgment back
Simpler but unreliable	More reliable
Initiator cannot confirm receipt	Both units confirm transfer success